"Capture the event. Update the state. Let React render the rest."

MODULE 34

State and Event Management in React

Learn how React manages state and events to build interactive, dynamic, and scalable user interfaces -- from useState fundamentals to controlled forms, lifted state, and effects.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 34 OVERVIEW

Manage component state and events in React so the UI stays in sync with user actions -- from useState to controlled forms and lifted state.

Every component eventually asks how its data changes and how the UI stays in sync -- this module answers that with useState, event handlers, controlled forms, and the lifted-state pattern that unifies a CRM dashboard's components.

DURATION & LAB

1 Hour Theory

State and hooks, event handling, controlled forms, and lifting state up to a shared parent.

2 Hours Lab

lab34-crm: lift CustomerList, CustomerForm, and CustomerToolbar into one App component.

Scenario

Freeze in-browser CRUD contracts so Lab 35 can swap fixtures for a real fetch client safely.

MODULE ARC

Capture the Event

Handle onClick, onChange, and onSubmit events the React way, without touching the DOM.

Update the State

useState's setter and immutable updates, lifted to a single App-level source of truth.

Build the Lab

Unify Lab 33's presentational CRM components into one stateful App with create, edit, cancel, and search.

KEY TAKEAWAY Total time: 3 hours (1 hour theory + 2 hours hands-on lab). By the end, you will manage state and events the React way across a real CRM component tree.

WHY STATE MANAGEMENT MATTERS

State is the heart of every interactive application. Proper state management ensures your UI is predictable, responsive, and scalable.

Drives Interactivity

State holds data that changes over time. When state updates, React efficiently re-renders and the UI instantly reflects it.

Ensures Consistency

Good state management keeps data consistent across components and prevents conflicting or outdated information.

Enables Scalability

As applications grow, well-structured state makes it easier to add features and maintain the codebase.

PROBLEMS WITHOUT PROPER STATE MANAGEMENT

Unpredictable UI

UI may not reflect the latest data or user actions correctly.

Hard to Debug

Scattered state and unclear data flow make bugs difficult to trace.

Poor Maintainability

Complex, duplicated state logic leads to harder maintenance.

Performance Issues

Unnecessary re-renders and inefficient updates slow the app down.

STATE UPDATE FLOW IN REACT

1. **User Action (Event)**
2. **State Updates**
3. **React Re-renders**
4. **UI is Updated**

KEY TAKEAWAY State management is not just about storing data -- it's about managing change, flow, and synchronization across your application.

Why State Management Matters

Effective state management is essential for building scalable, maintainable, and predictable React applications.

Single Source of Truth
Keeps application data in one central place.

Consistent UI
Ensures all components reflect the same data at all times.

Easier to Scale
Simplifies data flow as the application grows in complexity.

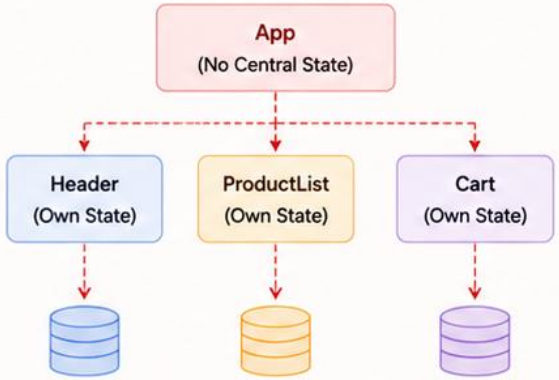
Fewer Bugs
Reduces errors caused by scattered or unsynchronized state.

Better Performance
Enables optimized updates and avoids unnecessary re-renders.

Maintainability
Makes code easier to understand, test, and maintain.

WITHOUT STATE MANAGEMENT

State is scattered across components, leading to inconsistent UI and hard-to-manage code.

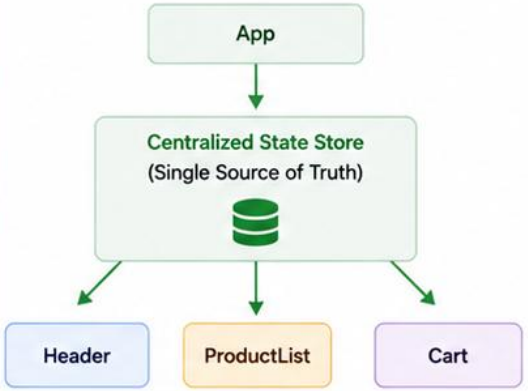


Problems

- Data duplication
- Inconsistent data across components
- Difficult to track state changes
- Hard to scale and maintain

WITH STATE MANAGEMENT

State is centralized and shared, ensuring consistency and predictable updates.



Benefits

- Single source of truth
- Consistent UI everywhere
- Predictable and traceable state updates
- Easier to scale and maintain

Key Takeaway

Good state management leads to predictable data flow, better performance, and a better developer experience.

- Predictable & Reliable
- Scalable Applications
- Clean & Maintainable Code
- Better Developer Experience

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to build interactive, responsive, and maintainable user interfaces by mastering state and event management in React.

- **Understand React State —** explain what state is and how it drives dynamic changes in a component.
- **Work with Local State —** create and manage local component state using the useState Hook.
- **Manage the State Lifecycle —** understand how state updates trigger re-renders and how React manages mount, update, and unmount.
- **Use React Hooks —** understand the role of useState, useEffect, and other Hooks and when to use them in functional components.
- **Handle Events —** handle user events (onClick, onChange, onSubmit) effectively to build interactive experiences.
- **Manage Forms and Inputs —** manage controlled form state and input values using React state and event handling.
- **Implement Data Flow Patterns —** implement parent-to-child (props) and child-to-parent (callback) communication.
- **Understand the State Update Workflow —** explain how state updates flow through React and result in a UI re-render.
- **Apply Enterprise UI Patterns —** use state management techniques -- lifting state up, decomposition, derived data -- to build complex, real-world interfaces.
- **Follow Best Practices —** apply React state and event management best practices for clean, immutable, maintainable code.

KEY TAKEAWAY Mastering state and event management is essential for building dynamic, interactive, and scalable React applications that deliver a great user experience.

WHAT IS STATE?

State is a built-in object in React components that holds data which can change over time. When it changes, React automatically re-renders the component.

1. **State Changes**
2. **React Detects**
3. **Component Re-renders**
4. **UI Updates**

KEY CHARACTERISTICS

- **Dynamic —** state data can change during the lifetime of a component.
- **Local to Component —** state belongs to the component where it is declared.
- **Triggers Re-render —** updating state causes React to re-render the component.
- **Managed by React —** React manages state values and the component lifecycle.

EXAMPLE -- COUNTER COMPONENT

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

Holds Data **Changes Over Time** **Triggers Re-render** **Updates UI**

KEY TAKEAWAY State is the heart of interactivity in React. It holds dynamic data and ensures your UI stays in sync with user actions and changes.

What is State?

State is a built-in React object that stores data that can change over time and affects how a component renders and behaves.

Dynamic Data
State holds data that can change during the lifecycle of a component.

Triggers Re-render
When state changes, React re-renders the component automatically.

Encapsulated
State is private to the component and cannot be accessed directly.

Persistent
State persists as long as the component is mounted in the UI.

Local to Component
State is managed within the component (unlike props, which are external).

EXAMPLE: COUNTER COMPONENT

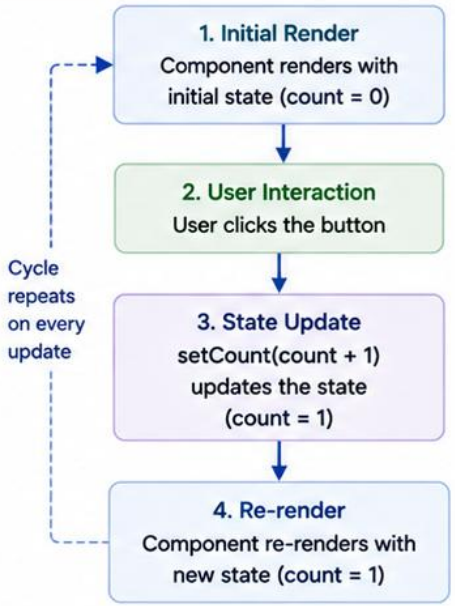
```jsx
import { useState } from 'react';

function Counter() {
   const [count, setCount] = useState(0);
    // count is the state variable
    // setCount is the function to update state

  return (
    <div className="counter">
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}

export default Counter;
```

HOW STATE WORKS



KEY POINTS

State Holds Changing Data
Used for data that changes over time, like user input, counters, toggles, etc.

Updating State Causes Re-render
When state changes, React updates the UI to reflect the new data.

Use State for Local, Component-Specific Data
If data is needed across components, use props, context, or external state management.

State Update is Asynchronous
State updates may not be immediate. React batches updates for performance.

LOCAL COMPONENT STATE

Local component state is data managed within a single component, using React's useState Hook -- private to that component and isolated from others.

1. **Initialize State**
2. **Update State**
3. **Re-render**
4. **UI Reflects Changes**

CHARACTERISTICS

- **Private —** state is accessible only within the component where it is declared.
- **Isolated —** changes in one component's state do not affect other components.
- **Triggers Re-render —** updating local state causes only that component to re-render.
- **Simple to Use —** ideal for UI-specific data like form inputs, toggles, and counters.

EXAMPLE -- ISOLATED COUNTER

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div className="counter">
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
export default Counter;
```

Owned by Component | **Private & Isolated** | **Triggers Re-render** | **Instant UI Updates**

KEY TAKEAWAY Use local state for data that belongs to a single component and does not need to be shared with others.

STATE LIFECYCLE

The state lifecycle describes how state is created, updated, and destroyed during the life of a React component.

1. **Mount**
   Component added to DOM
2. **Initialize**
   State set with useState
3. **Update**
   State changes, re-renders
4. **Unmount**
   Removed, cleanup runs

WHAT HAPPENS IN EACH PHASE

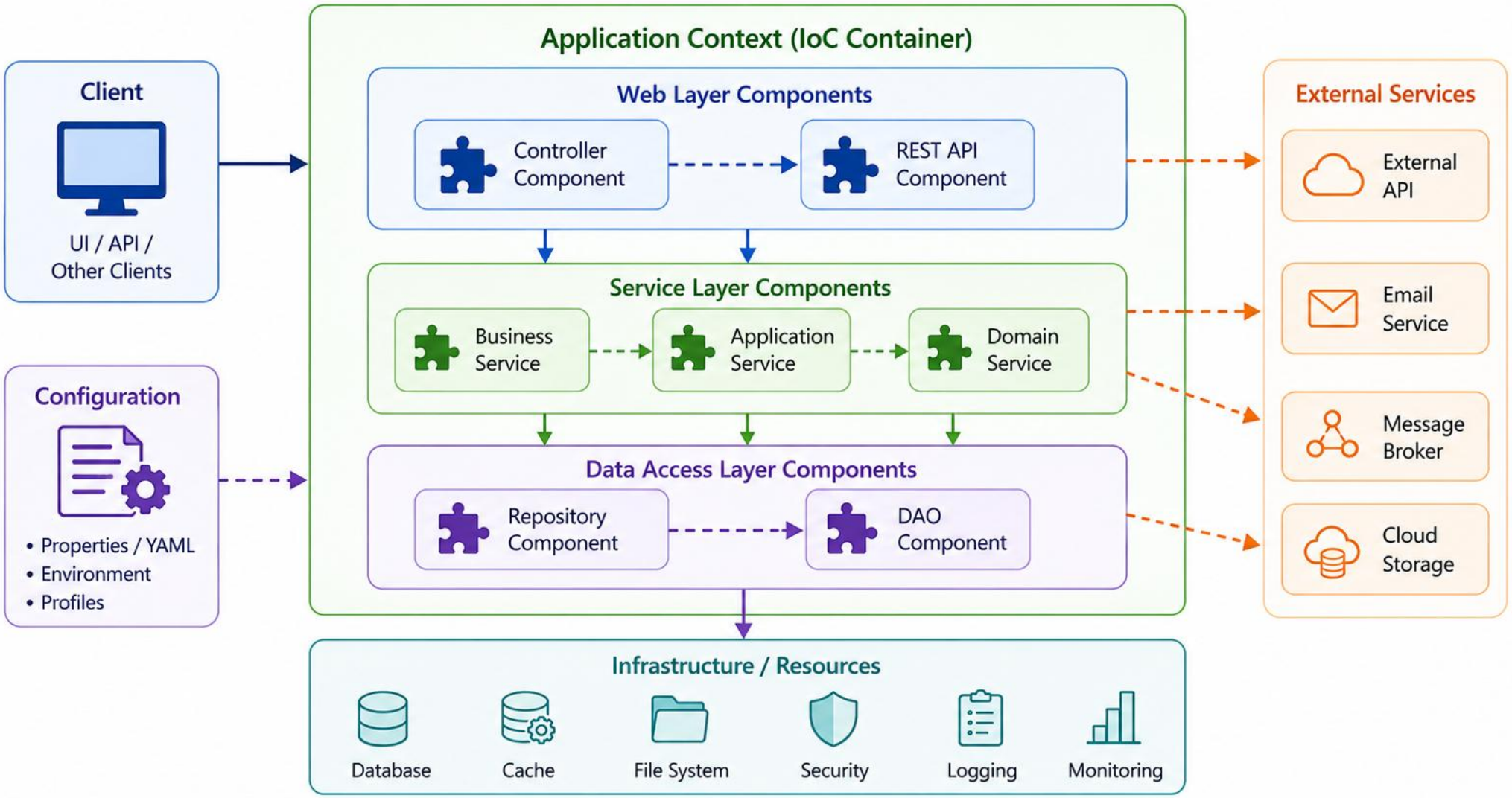
- **Mount —** the component is inserted into the DOM.
- **Initialize —** initial state is set using useState.
- **Update —** state updates cause the component to re-render with new values.
- **Unmount —** the component is removed from the DOM; cleanup logic (if any) runs.

EXAMPLE -- MOUNT / UNMOUNT WITH useEffect

```
function Demo() {
  const [count, setCount] = useState(0);
  useEffect(() => {
    console.log('Component Mounted');
    return () => {
      console.log('Component Unmounted');
    };
  }, []); // runs on mount and unmount
  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

KEY TAKEAWAY Understanding the state lifecycle helps you write predictable, efficient, and bug-free React components.

Component Composition



INTRODUCTION TO REACT HOOKS

React Hooks are functions that let you use state and other React features without writing a class -- making components simpler and easier to understand.

WHY HOOKS?

- **Use React Features —** in functional components, without ever writing a class.
- **Reuse Logic —** share stateful behavior across components with custom Hooks.
- **Less Code —** cleaner and more readable than class-based lifecycle methods.
- **Better Organization —** related logic stays grouped together instead of split across lifecycle methods.

BUILT-IN HOOKS YOU WILL USE

Hook	What It Does
useState	Adds state to a functional component.
useEffect	Performs side effects (fetch, subscriptions, timers) in a function component.
useContext	Accesses context values without a Consumer.
useRef	Stores a mutable value that does not cause a re-render.

RULES OF HOOKS

- **Only call Hooks at the top level —** never inside loops, conditions, or nested functions.
- **Only call Hooks from React functions —** functional components or custom Hooks, not plain JavaScript functions.

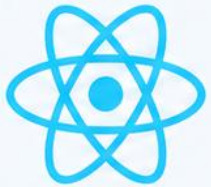
HOOKS VS. CLASSES

Less boilerplate | **Easier logic reuse**

Simpler lifecycle (useEffect) | **Easier to test**

KEY TAKEAWAY Hooks let you use state and other React features in function components -- in a simpler, more flexible way than class components ever offered.

Introduction to React Hooks



React Hooks are functions that let you “hook into” React features like state and lifecycle from functional components.

- Use state and other React features
- Reuse logic between components
- Write cleaner, simpler and more maintainable code

Why Hooks?

Simpler Components
Use state and lifecycle features without writing class components.

Reusability
Extract and reuse stateful logic with Custom Hooks.

Better Readability
Organize related logic together inside functional components.

Easier Maintenance
Less boilerplate and more focused, predictable code.

Common Built-in Hooks

useState	Adds state to functional components.
useEffect	Performs side effects in function components (e.g., API calls, subscriptions).
useContext	Access context values without nesting components.
useRef	Persists a mutable value across re-renders without causing re-render.
useReducer	Manages complex state logic with a reducer function.
useMemo	Memorizes a computed value to optimize performance.
useCallback	Memorizes a function to prevent unnecessary re-creations.

How Hooks Work

1. Import Hook
Import the hook you need from React.

2. Call Hook
Call the hook at the top level of your functional component.

3. Use Returned Values
Use the state, functions, or values returned by the hook.

4. React Handles Updates
React automatically manages updates and re-renders when needed.

USESTATE HOOK

Syntax: const [state, setState] = useState(initialState); -- the most commonly used Hook in React, letting you add state to functional components.

HOW useState WORKS

INITIAL STATE CAN BE ANY TYPE

Type	Example
Number	useState(0)
String	useState('Aman')
Boolean	useState(false)
Object	useState({ id: 1, name: 'Aman' })
Array	useState([])

EXAMPLE

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div className="counter">
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

KEY POINTS

- State is local to the component where it is declared.
- Updating state does not change the value immediately in the current render.
- React batches state updates together for better performance.
- Never mutate state directly -- always call the setter function.

KEY TAKEAWAY useState is your go-to Hook for managing local state in functional components. It makes components simple, interactive, and easy to maintain.

useState Hook

useState is a React Hook that lets you add state to functional components.

- Adds state to functional components
- Triggers re-render when state updates
- Returns current state and a function to update it

Syntax

```
const [state, setState] = useState(initialState);
```

- **state** — The current state value.
- **setState** — The function used to update the state. Updating state causes the component to re-render.

Example

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

How it Works

Component Renders
useState returns the current state and the setState function.

↓

State Update
When setState is called, React updates the state.

↓

Re-render
React re-renders the component with the new state.

↓

UI Updated
The updated state is reflected in the UI.

Key Points

- useState can be used multiple times in a single component.
- State updates are asynchronous.
- Initial state is used only on the first render.
- You can pass a function to setState when the new state depends on previous state.

Tip
Use descriptive state names and keep state as simple as possible.

TRY IT YOURSELF — EXERCISE 1: FILL USESTATE TODOS (STARTER)

Reinforces: completing the useState/onChange TODO blanks in a tiny CRM form snippet — a hands-on starter (not a finished solution).

STARTER SNIPPET (notes/lab34-todos.md) — FILL EVERY ____ AND // TODO

```
const [name, setName] = useState(_____);
const [status, setStatus] = useState<_____>(_____);
const [error, setError] = useState<string | null>(null);

function onSubmit(e: FormEvent) {
  e.preventDefault();
  if (!name.trim()) { setError(_____); return; }
  // TODO Lab 35: POST to API
  console.log({ name, status, correlation: "lab-request-001" });
}

<input value={name} onChange={(e) => _____(e.target.value)} />
```

STEPS (IN notes/lab34-todos.md)

Step	What To Do
1 — Paste	Create notes/lab34-todos.md and paste the starter snippet above exactly as shown.
2 — Fill the Blanks	Suggested values: "", 'ACTIVE' | 'SUSPENDED', "ACTIVE", "Name is required", setName.
3 — Amina Seed (Optional)	Alternate: initialize name to "Amina Khan" and add a // TODO edit-form comment.
4 — Lift-State Note	Add a // TODO: selectedCustomerId lives in the parent list container, not this form.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 1 (STARTER) before continuing: exercises/exercise-01-fill-usestate-todos.md (workspace: examples/module-34-exercises).

USEEFFECT HOOK

useEffect lets you perform side effects in function components -- it runs after render and is used for data fetching, subscriptions, timers, and DOM updates.

HOW useEffect WORKS

1. **Render**
2. **Run Effect**
3. **Check Dependencies**
4. **Cleanup (Optional)**

COMMON DEPENDENCY PATTERNS

Deps	When It Runs	Use Case
[]	Once, on mount	Initial fetch, setup subscriptions/timers
[value]	When value changes	Re-fetch or re-sync on prop/state change
(none)	After every render	Rarely needed -- can cause performance issues

EXAMPLE -- DATA FETCHING WITH CLEANUP

```
useEffect(() => {
  let active = true;
  setLoading(true);
  fetch('/api/users')
    .then((res) => res.json())
    .then((data) => { if (active) setUsers(data); })
    .finally(() => setLoading(false));
  return () => { active = false; }; // cleanup
}, []); // empty array -> run once on mount
```

BEST PRACTICES

- **Always specify dependencies**
- **Keep effects focused**
- **Clean up subscriptions/timers**
- **Avoid effects for derived data**

KEY TAKEAWAY useEffect synchronizes your component with the outside world by running side effects at the right time -- always clean up what you start.

useEffect Hook

useEffect is a React Hook that lets you perform side effects in functional components.

- Perform side effects like API calls, subscriptions, or manual DOM updates
- Runs after the component renders
- Supports cleanup to avoid memory leaks

Syntax

```
useEffect(() => {
  // side effect logic
  return () => {
    // cleanup logic (optional)
  };
}, [dependencies]);
```

Example

```
import { useState, useEffect } from 'react';

function User() {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch('https://api.example.com/user/1')
      .then(res => res.json())
      .then(data => setUser(data));
  }, []); // runs once on mount

  return (
    <div>{user ? user.name : 'Loading...'}</div>
  );
}
```

How it Works

Component Renders
React renders the component to the screen.

↓

Effect Runs
After render, React runs the code inside useEffect.

↓

Dependencies Check
React compares dependencies with the previous values.

- **No Change** — Effect does not run again.
- **Changed** — Cleanup runs (if any), then effect runs again.

Cleanup (Optional)
Cleanup function runs before the component unmounts or before the next effect run.

Dependency Array

`[]`	Runs only once after the first render (componentDidMount).
`[dep]`	Runs on mount and whenever *dep* changes.
`[dep1, dep2]`	Runs on mount and whenever any dependency changes.
`No Array`	Runs after every render (not recommended).

Common Use Cases

- Fetching data from APIs
- Setting up subscriptions or event listeners
- Manually updating the DOM
- Timers and intervals
- Logging and analytics

HANDLING USER EVENTS (1 OF 2)

React makes it easy to handle user interactions like clicks, key presses, form inputs, and more using event handlers.

WHAT IS AN EVENT?

An event is an action triggered by the user or the browser, such as clicking a button, typing in a field, or moving the mouse.

COMMON USER EVENTS IN REACT

- **onClick**
- **onChange**
- **onSubmit**
- **onKeyDown**
- **onMouseOver**

HOW EVENT HANDLING WORKS

1. **User Action**
2. **Event Triggered**
3. **Handler Runs**
4. **UI Updates**

EXAMPLE -- BUTTON CLICK EVENT

```
import { useState } from 'react';

function ClickCounter() {
  const [count, setCount] = useState(0);

  // Event handler function
  const handleClick = () => {
    setCount(count + 1);
    console.log('Button clicked!');
  };

  return (
    <div className="container">
      <h2>Click Count: {count}</h2>
      <button onClick={handleClick}>Click Me</button>
    </div>
  );
}
```

KEY TAKEAWAY Handling user events is essential for creating interactive applications. React's event system is simple, consistent, and powerful.

Handling User Events

React makes it easy to handle user interactions like clicks, input changes, form submissions, and more.

- Respond to user actions
- Update state accordingly
- Re-render UI
- Create dynamic experiences

Common Events

onClick
Triggered when an element is clicked.

onChange
Triggered when the value of an input, select, or textaraa changes.

onSubmit
Triggered when a form is submitted.

onFocus / onBlur
Triggered when an element gets focus / loses focus.

onMouseOver / onMouseOut
Triggered when the mouse enters / leaves an element.

Basic Example

```
import { useState } from 'react';
function ButtonExample() {
  const [count, setCount] = useState(0);
  const handleClick = () => {
    setCount(count + 1);
  };
  return (
    <div>
      <h3>You clicked {count} times</h3>
      <button onClick={handleClick}>
        Click Me
      </button>
    </div>
  );
}
```

How It Works

User clicks the button → handleClick() is called → State (count) updates → Component re-renders → Updated UI is shown

Event in JSX

```
<button onClick={handleClick}>Click</button>
<input type="text" onChange={handleChange} />
<form onSubmit={handleSubmit}>...</form>
```

Key Points

- Event names in React use camelCase.
- Pass a function as the event handler.
- State updates inside handlers trigger UI re-render.
- You can pass arguments using arrow functions.
- React events are synthetic events for better performance.

HANDLING USER EVENTS (2 OF 2)

Event handler basics, passing data to handlers, preventing default behavior, and where common events are used across form elements.

PASS THE REFERENCE, NOT THE CALL

```
// Correct -- pass the reference
<button onClick={handleClick}>
  Click Me
</button>
```

```
// Incorrect -- calls immediately during render
<button onClick={handleClick()}>
  Click Me
</button>
// this runs on every render, not on click
```

COMMON EVENTS BY ELEMENT

Element	Common Events	Purpose
Button	onClick	Handle button clicks
Input	onChange, onFocus, onBlur	Handle input changes, focus, blur
Form	onSubmit	Handle form submission

PASSING DATA TO EVENT HANDLERS

```
const handleDelete = (id) => {
  console.log('Delete item:', id);
};

<button onClick={() => handleDelete(42)}>Delete</button>
```

PREVENTING DEFAULT BEHAVIOR

```
const handleSubmit = (e) => {
  e.preventDefault(); // stop page refresh
  console.log('Form submitted!');
};

<form onSubmit={handleSubmit}>
  <button type="submit">Submit</button>
</form>
```

KEY TAKEAWAY Always pass the function reference, not the function call; use an arrow function to pass arguments; call preventDefault() to stop the default browser behavior.

TRY IT YOURSELF — EXERCISE 2: EVENT HANDLER MAP

Reinforces: documenting which handler updates which piece of CRM UI state — a documentation exercise.

HANDLER MAP — EVENT / HANDLER / STATE UPDATED

Event	Handler	State Updated
Name input onChange	handleNameChange	draft.name (draft/form state)
Status select onChange	handleStatusChange	draft.status (draft/form state)
Form onSubmit	handleSubmit	validate() then customers (create/update)
Customer row onClick	handleSelect(id)	mode -> { kind: 'edit', id } (select Amina, CUS-1001)

STEPS (IN notes/lab34-events.md)

- **Step 1 — Table —** use exactly these three columns: Event, Handler, State Updated.
- **Step 2 — Rows —** include at least the four rows above -- two field onChange rows, one form onSubmit row, and one row onClick -> select Amina.
- **Step 3 — Derived —** add a note that isValid is DERIVED from draft/errors during render, not stored in its own useState.
- **Step 4 — Save —** keep the finished table in your notes; Lab 34 reuses this exact handler naming.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-event-handler-map.md (workspace: examples/module-34-exercises).

FORMS AND FORM STATE (1 OF 2)

Forms let users enter data and interact with your application. In React, form elements are controlled using component state, so the UI always reflects the latest values.

WHY USE CONTROLLED COMPONENTS?

- **Single Source of Truth —** state, not the DOM, always holds the current input value.
- **Validation & Error Handling —** every keystroke can be checked before submit.
- **Easier to Manage and Test —** form values are plain data, not DOM queries.
- **UI Always in Sync —** the displayed value always matches component state.

KEY PRINCIPLES

- **Use value to bind —** an input's value prop always comes from state, never from the DOM.
- **Use onChange to update —** every keystroke updates state through the setter.
- **Give every input a name —** so one handler can identify which field changed.
- **Call preventDefault() on submit —** to stop the page from reloading.

HOW FORMS WORK IN REACT

EXAMPLE -- CONTROLLED FORM

```
function UserForm() {
  const [form, setForm] = useState({ name: '', email: '' });

  const handleChange = (e) => {
    const { name, value } = e.target;
    setForm({ ...form, [name]: value });
  };
  const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Submitted:', form);
  };
  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={form.name} onChange={handleChange} />
      <input name="email" value={form.email} onChange={handleChange} />
      <button type="submit">Submit</button>
    </form>
  );
}
```

KEY TAKEAWAY Controlled components keep form state as the single source of truth -- bind value to state, update it via onChange, and prevent default on submit.

Forms and Inputs

In React, form elements and inputs are controlled by component state, making data handling predictable and efficient.

Controlled Components
Form data is handled by React state

Real-time Updates
UI updates as the user types or selects

Validation Ready
Easy to add validation and error handling

Reusable Logic
Build reusable form components

Common Form Elements

Text Input
`<input type="text" />`
Single-line text input

Email Input
`<input type="email" />`
Email address input

Password Input
`<input type="password" />`
Password input

Textarea
`<textarea />`
Multi-line text input

Select
`<select> <option> </option> </select>`
Dropdown selection

Checkbox
`<input type="checkbox" />`
Multiple choice (true/false)

Radio Button
`<input type="radio" />`
Single choice from options

Controlled Input Example

```
import { useState } from 'react';
function MyForm() {
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    message: ''
  });

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData({ ...formData, [name]: value });
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log(formData);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input type="text" name="name" value={formData.name}
             onChange={handleChange} placeholder="Your name" />
      <input type="email" name="email" value={formData.email}
             onChange={handleChange} placeholder="Your email" />
      <textarea name="message" value={formData.message}
              onChange={handleChange} placeholder="Message" />
      <button type="submit">Send</button>
    </form>
  );
}
```

How It Works

User types or selects
User interacts with the form field

↓

onChange fires
Updates React state with new value

↓

State updates
React re-renders the component

↓

UI stays in sync
The input value reflects the state

Key Points

- Use state to store form values.
- Bind value and onChange to inputs.
- Handle form submission with onSubmit.
- Easy to validate and show errors.
- Makes forms predictable and testable.

FORMS AND FORM STATE (2 OF 2)

A quick reference for common input types, the single-state-object pattern for multi-field forms, and best practices for managing form state.

Input Type	Example	Handling
text	John Doe	value + onChange
email	john@example.com	value + onChange
password	••••••••	value + onChange (masked)
number	25	value + onChange (numeric)
checkbox	☑ I agree	checked + onChange
radio	◉ Male ○ Female	checked + onChange
select	Canada ▾	value + onChange

BEST PRACTICES

- **Keep form state minimal**
- **Provide instant feedback**
- **Validate on change or submit**
- **Reset after successful submission**

COMMON PATTERN: SINGLE STATE OBJECT

```
const [form, setForm] = useState({
  name: '',
  email: '',
  password: '',
});
```

IMPORTANT TIPS

- Always use the name attribute for each input.
- Use onChange to keep state in sync with user input.
- Avoid using refs for form state (except uncontrolled forms).
- Validate before submitting the form.

KEY TAKEAWAY Keep state as the single source of truth: prefer one state object for related fields, and validate user input before submitting.

Managing Form State

In React, form state is stored in the component using **useState** and updated as the user interacts with the inputs.

Single Source of Truth
All form data lives in React state

Real-time Sync
Inputs and state stay in sync on every change

Predictable Updates
State updates trigger re-renders

Easy Validation
Validate and handle errors efficiently

Form State with useState

Declare State
Use useState to store all form field values.

Update State
Update state on every input change.

Read State
Access state values on submit or for validation.

Benefits

- Centralized form data management
- Instant updates and feedback
- Easier validation and error handling
- Scalable for complex forms

Example

```jsx
import { useState } from 'react';

function ContactForm() {
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    message: ''
  });

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData({ ...formData, [name]: value });
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log(formData); // { name, email, message }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input type="text" name="name" value={formData.name}
             onChange={handleChange} placeholder="Your name" />
      <input type="email" name="email" value={formData.email}
             onChange={handleChange} placeholder="Your email" />
      <textaraa name="message" value={formData.message}
              onChange={handleChange} placeholder="Message" />
      <button type="submit">Send</button>
    </form>
  );
}
export default ContactForm;
```

How It Works

User types in the input field
Enters data in any form field.

onChange

onChange is triggered
The handleChange function runs.

State is updated
setFormData updates the form data in state.

Component re-renders
The UI reflects the latest state.

Data is ready to use
Use the state on submit or for validation.

Best Practices

- Use meaningful field names.
- Keep state shape flat and simple.
- Validate on change or on submit.
- Show user-friendly error messages.

TRY IT YOURSELF — EXERCISE 3: CONTROLLED FORM SKETCH

Reinforces: sketching the controlled-input flow for a new customer, Ravi Singh (CUS-1002) — an architecture exercise (paper, no code yet).

REFERENCE — UI PIECE TO STATE FIELD

UI Piece	State Field
Name input	name
Status select	status
Error text	error
Submit disabled	isValid (derived, not stored)

CONTROLLED-FORM FLOW (NUMBER THE STEPS)

1 **Render**

2 **onChange Updates State**

3 **Validate**

4 **onSubmit**

FIXTURE — RAVI SINGH DRAFT (CUS-1002)

```
const draft = { name: "Ravi Singh", status: "ACTIVE" };
// render -> onChange updates state -> validate -> onSubmit
// submit assigns CUS-1002 (server assigns the id for real in Lab 35)
// Note: uncontrolled refs are OUT OF SCOPE for this lab path
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 3 before continuing: exercises/exercise-03-controlled-form.md (workspace: examples/module-34-exercises).

TRY IT YOURSELF — EXERCISE 4: VALIDATION MESSAGES

Reinforces: drafting user-facing validation copy for the CRM form's required fields — a documentation exercise.

VALIDATION RULES FOR THE CRM FORM

Rule	Your Task
Name is required	Draft a user-facing message -- no jargon.
Status is required	Draft a user-facing message -- no jargon.
Name minimum length 2	Draft a user-facing message -- no jargon.

STEPS (IN notes/lab34-validation.md)

- **Step 1 — Rules —** name required; status required; name minimum length 2.
- **Step 2 — Messages —** draft three user-facing strings, plain language, no technical jargon.
- **Step 3 — Timing —** choose validate-on-blur vs. validate-on-submit -- pick ONE for Lab 34.
- **Step 4 — Server Later —** note that Lab 35 will ALSO show API 400 errors alongside this client validation.

EXAMPLE — DISPLAYING A FIELD ERROR

```
{errors.name && (
  <span className="field-error">{errors.name}</span>
)}
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-validation-messages.md (workspace: examples/module-34-exercises).

PARENT-TO-CHILD COMMUNICATION

In React, data flows down from parent components to their children using props -- a one-way, predictable data flow.

HOW IT WORKS

1. **State in Parent**
2. **Pass as Props**
3. **Receive in Child**
4. **Render with Data**

KEY TAKEAWAYS

- Parents own the state and pass it to children using props.
- Props are read-only in the child -- a child cannot modify them.
- Data flows down only, one way, keeping the pattern predictable.
- This keeps components simple, reusable, and easy to debug.

EXAMPLE -- APP.TSX PASSES NAME TO GREETING

```
function App() {
  const [name, setName] = useState('Aman');
  return (
    <div>
      <input value={name} onChange={(e) => setName(e.target.value)} />
      <Greeting name={name} />
    </div>
  );
}

function Greeting({ name }) {
  return <h3>Hello, {name}!</h3>; // read-only prop
}
```

Prop Type	Example
String	<Greeting name="Aman" />
Number	<Counter count={10} />
Object / Array	<Customer data={record} />
Function	<Button onClick={handleClick} />

KEY TAKEAWAY Props are read-only and flow one way -- from parent to child. Pass only what a child needs, and keep components focused.

Parent-to-Child Communication

In React, parent components share data with child components by passing **props**.

- **One-way data flow** — Data flows from parent to child
- **Props** — Parent passes data as props to child
- **Read-only** — Child reads props but does not modify them
- **Re-render** — Child re-renders when props change

How It Works

Parent Component
Holds the data (state)

```
const data = "Hello";
```

Pass as Props
Send data to child using props

```
<ChildComponent message={data} />
```

Child Component
Receives props and uses them for rendering

```
props.message
```

Key Point
Props are read-only in the child component.

Example

```jsx
// ParentComponent.js
import React from 'react';
import ChildComponent from './ChildComponent';

function ParentComponent() {
  const user = { name: 'Alice', age: 25 };
  return (
    <div>
      <h2>Parent Component</h2>
      <ChildComponent user={user} />
    </div>
  );
}
export default ParentComponent;
```

```jsx
// ChildComponent.js
import React from 'react';

function ChildComponent(props) {
  return (
    <div>
      <h3>Child Component</h3>
      <p>Name: {props.user.name}</p>
      <p>Age: {props.user.age}</p>
    </div>
  );
}
export default ChildComponent;
```

Output (UI)

Parent Component

Child Component

Name: Alice

Age: 25

Key Takeaways

- Parent passes data to child via props.
- Child receives props as function arguments.
- Data flows in one direction (parent → child).
- If props change, child re-renders automatically.
- Child cannot modify props.

CHILD-TO-PARENT COMMUNICATION

In React, data flows up from child components to their parents using callback props -- children call a function passed down by the parent to send data up.

HOW IT WORKS

1. **Define Callback**
2. **Passed as Prop**
3. **Child Calls Callback**
4. **Parent Updates State**

EXAMPLE -- CHILD SENDS A MESSAGE TO APP

```
function App() {
  const [message, setMessage] = useState('');
  const handleMessage = (msg) => setMessage(msg);
  return <Child onSendMessage={handleMessage} />;
}

function Child({ onSendMessage }) {
  const [text, setText] = useState('');
  const handleClick = () => {
    onSendMessage(text); // send data up
    setText('');
  };
  return <button onClick={handleClick}>Send to Parent</button>;
}
```

KEY TAKEAWAYS

- Children cannot change the parent's state directly.
- Instead, they call a function (callback) passed down as a prop.
- Data flows up; the parent decides what to do with it.
- This keeps components decoupled, reusable, and testable.

COMMON USE CASES

- **Forms: input -> parent state**
- **Lists: action -> parent updates list**
- **Filters: selection -> parent applies filter**

KEY TAKEAWAY Child-to-parent communication uses callback props: the parent defines a function, passes it down, and the child calls it to send data back up.

Child-to-Parent Communication

In React, children send data to their parent components by passing **callback functions** as props.

- **Bottom-up data flow** — Data flows from child to parent
- **Callback Props** — Parent passes a function to child
- **Event Triggered** — Child calls the function on events
- **Parent State Updates** — Parent updates its state and re-renders

How It Works

Parent Component
Defines a state and a callback function that updates the state.

```
const handleData = (data) => { ... }
```

Pass Callback to Child
The parent passes the callback function to the child via props.

```
<ChildComponent onData={handleData} />
```

Child Component
Calls the callback function and sends data to the parent.

```
props.onData(value)
```

Parent Receives Data
The parent receives the data, updates its state, and re-renders the UI.

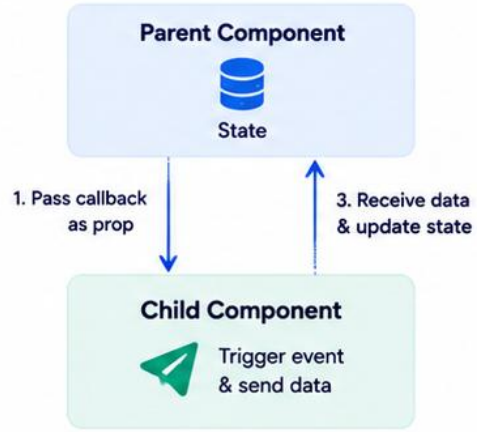
Example

```jsx
// ParentComponent.js
import React, { useState } from 'react';
import ChildComponent from './ChildComponent';

function ParentComponent() {
  const [message, setMessage] = useState('');
  const handleData = (data) => {
    setMessage(data);
  };
  return (
    <div>
      <h2>Parent Component</h2>
      <ChildComponent onData={handleData} />
      <p><strong>Message from child:</strong> {message}</p>
    </div>
  );
}
export default ParentComponent;
```

```jsx
// ChildComponent.js
import React, { useState } from 'react';
function ChildComponent(props) {
  const [input, setInput] = useState('');
  const sendData = () => {
    props.onData(input); // send data to parent
 };
 return (
   <div>
     <h3>Child Component</h3>
     <input value={input} onChange={(e) => setInput(e.target.value)} />
     <button onClick={sendData}>Send to Parent</button>
   </div>
  );
}
export default ChildComponent;
```

Flow Diagram



Key Takeaways

- Child sends data to parent using callback props.
- Parent defines the function and passes it to child.
- Child calls the function with the data.
- Parent updates state and re-renders the UI.
- This enables dynamic and interactive UIs.

TRY IT YOURSELF — EXERCISE 5: PROPS VS STATE

Reinforces: classifying prop vs. state fields on Amina Khan's (CUS-1001) customer edit form — an analysis exercise.

THE SCENARIO (CustomerEditForm.tsx) — EDITING AMINA (CUS-1001)

```
function CustomerEditForm({ initialCustomer, onSaved }) {
  const [draftName, setDraftName] = useState(initialCustomer.name);
  const [draftStatus, setDraftStatus] = useState(initialCustomer.status);
  const [isSaving, setIsSaving] = useState(false);
  // Classify each of the 5 identifiers above: prop, or state?
}
```

STEPS (IN notes/lab34-state.md)

Step	What To Do
1 — Scenario	Editing Amina (CUS-1001): name field, status dropdown, Save button.
2 — Classify	Mark each as prop or state: initialCustomer, draftName, draftStatus, isSaving, onSaved callback.
3 — Rule	Write the rule sentence: "state = data that changes over time because of user interaction in this component."
4 — Notes	Save your classification table and rule sentence to notes/lab34-state.md.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 5 before continuing: exercises/exercise-05-props-vs-state.md (workspace: examples/module-34-exercises).

STATE UPDATE WORKFLOW

When state is updated, React goes through a well-defined, asynchronous workflow: request the change, React updates and re-renders, and the UI reflects the new state.

1. **Event Triggered**
2. **setState Called**
3. **Update Queued**
4. **State Updated**
5. **Component Re-renders**
6. **UI Updated**

WHAT HAPPENS BEHIND THE SCENES

- React does not update the DOM immediately when a setter is called.
- It batches the update and applies it after the current event handler finishes.
- Then React re-renders the component with the new state.
- Finally, React updates only the parts of the real DOM that actually changed.

EXAMPLE -- COUNTER.TSX

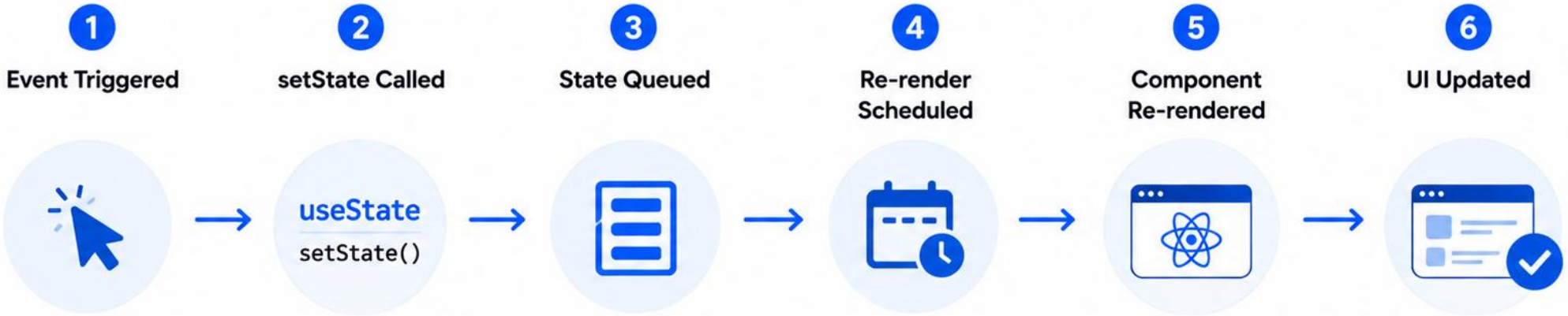
```
const [count, setCount] = useState(0);
const handleIncrement = () => {
  setCount(count + 1); // request state update
};
```

BEST PRACTICE -- FUNCTIONAL UPDATES

```
setCount((prev) => prev + 1); // always up to date
```

KEY TAKEAWAY State updates are asynchronous and batched -- use functional updates (prev => prev + 1) whenever the new state depends on the previous state.

State Update Workflow



User interacts with the UI (e.g., click, input change).	Event handler calls setState() to update the state.	React queues the state update and waits for the current code to finish.	React schedules a re-render of the component.	Component function runs again with the updated state.	React updates the DOM and the UI reflects the new state.

Key Points

- State updates are asynchronous.
- React may batch multiple state updates for better performance.
- Component re-renders only when state or props change.
- UI always reflects the latest state.

MANAGING COMPLEX USER INTERFACES (1 OF 2)

As applications grow, UIs become complex with many interactive parts and shared state. React provides patterns to organize state and stay maintainable.

CHALLENGES IN COMPLEX UIs

- Many components and interactions to coordinate.
- Shared state needed across different parts of the UI.
- Deeply nested component trees.
- Managing forms, lists, filters, and pagination together.
- Keeping the UI fast and responsive as it grows.

KEY IDEA

Break the UI into smaller, reusable pieces, manage state effectively, and ensure data flows in the right direction.

COMPONENT DECOMPOSITION EXAMPLE

Dashboard (Page) -> Sidebar, Header, Main Content, User Profile -> Main Content splits into Stats Cards, Chart, Recent Activity. Break the interface into reusable, manageable components.

STRATEGIES FOR MANAGING COMPLEX UIs

Strategy	What It Means
Component Decomposition	Break the UI into small, focused components with a single responsibility.
Lift State Up	Store shared state in the nearest common parent and pass it down via props.
Custom Hooks	Extract and reuse stateful logic across multiple components.
Context API	Share state across deeply nested components without prop drilling.
State Management Libraries	Use tools like Redux Toolkit, Zustand, or Recoil for large-scale needs.
Performance Optimization	Use memoization, lazy loading, and virtualization for better performance.

KEY TAKEAWAY Break the UI into smaller, reusable pieces, manage state effectively, and ensure data flows in the right direction.

Managing Complex User Interfaces

Break down complex UIs into maintainable, scalable, and reusable pieces.

Component Decomposition
Build small, focused, reusable components

State Management
Manage state efficiently and predictably

Data Flow
Unidirectional data flow keeps UI consistent

UI Patterns
Use proven patterns for better UX and code quality

Performance
Optimize rendering and reduce re-renders

Best Practices

Component Composition
Compose complex UIs from small, reusable components.

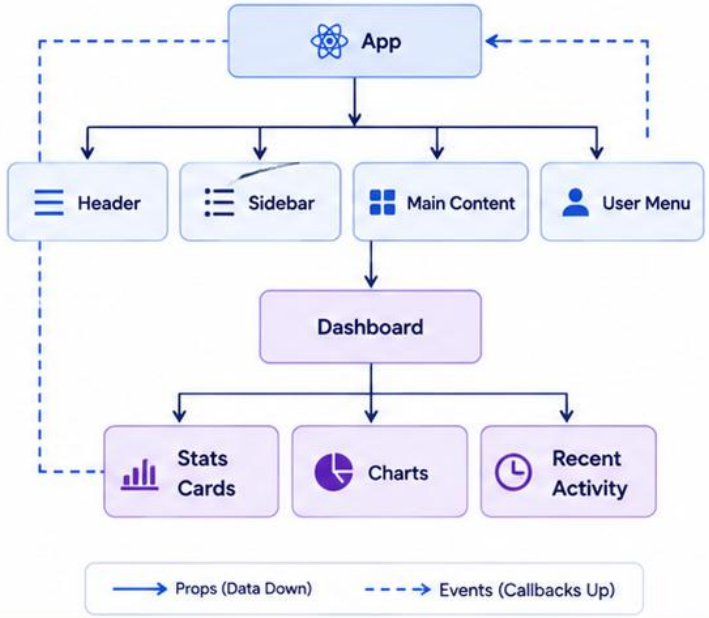
Lift State Up
Share state at the nearest common ancestor.

Unidirectional Data Flow
Data flows down via props, events flow up via callbacks.

Custom Hooks
Extract and reuse stateful logic with custom hooks.

Memoization
Use React.memo, useMemo, useCallback to prevent unnecessary re-renders.

Example Architecture



State Management Options

Local State (useState / useReducer)
Good for component-specific state.

Context API
Share state across the component tree without prop drilling.

Redux / Zustand / Jotai
Ideal for large-scale applications with complex global state.

Common UI Patterns

Tabs
Organize content into manageable sections.

Modals & Dialogs
Display important content or get user input.

Accordions
Show and hide content in a compact way.

Steppers
Guide users through multi-step processes.

Key Takeaway
Structure your UI, manage state wisely, and follow best practices to build complex interfaces that are maintainable, performant, and scalable.

- Think component-first
- Keep state minimal and normalized
- Optimize for performance
- Focus on a great user experience

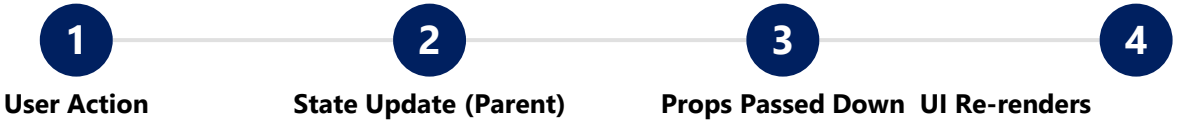
MANAGING COMPLEX USER INTERFACES (2 OF 2)

Choosing the right pattern for the job, and the unidirectional data-flow discipline that keeps large React applications predictable.

WHEN TO USE WHAT?

Need	Recommended Approach
Share state between nearby components	Lift state up
Share state across a deep component tree	Context API
Reuse stateful logic across components	Custom Hooks
Manage complex, app-wide global state	State management library
Improve rendering performance	Memoization, code splitting, virtualization

DATA FLOW IN COMPLEX UIs



Unidirectional data flow keeps your UI predictable and easier to debug -- state is updated in one place and flows down through props.

BEST PRACTICES

- **Keep components small & focused**
- **Keep state close to where it's used**
- **Derive data instead of duplicating**
- **Test interactions thoroughly**

KEY TAKEAWAY Managing complex UIs is about structuring components, managing state wisely, and optimizing performance -- keep data flow unidirectional and predictable.

REACT STATE BEST PRACTICES

Following these best practices helps you write predictable, maintainable, and performant React applications. Golden rule: keep state minimal and close to where it's used.

#	Practice	What It Means
1	Keep State Local When Possible	Store state at the lowest common ancestor to reduce complexity and unnecessary re-renders.
2	Avoid Redundant or Derived State	Don't store data in state that can be calculated from props or other state -- derive it during render.
3	Use State Updates Immutably	Never mutate state directly; always use the setter and return new values (spread, map, filter).
4	Use Functional Updates When Next State Depends on Previous	Pass a function to the setter to get the latest value and avoid stale-closure bugs.
5	Don't Over-Lift State	Avoid lifting state higher in the tree than necessary -- it makes state harder to track and reuse.
6	Keep State Shape Simple and Flat	Prefer simple structures over deeply nested objects to simplify updates.
7	Avoid Large or Complex State	Split large state into smaller, focused pieces to improve readability and performance.
8	Clean Up State When No Longer Needed	Remove obsolete state and reset it on unmount if needed to prevent memory leaks.

Do	Don't
Use meaningful state names. Keep components focused. Use controlled components for forms. Use useReducer or Context for complex state logic.	Don't mutate state directly. Don't store non-serializable values in state unless necessary. Don't duplicate data in multiple places. Don't ignore React warnings about state updates.

KEY TAKEAWAY Golden Rule: keep state minimal, meaningful, and close to where it is used. When in doubt, simplify.

React State Best Practices

Follow these best practices to write predictable, maintainable, and high-performance React applications.

- **Predictable** — State changes are easy to reason about
- **Maintainable** — Clean and organized state logic
- **Performant** — Avoid unnecessary re-renders
- **Scalable** — Easily adapt to growing complexity
- **Reliable** — Fewer bugs and side effects

1 Keep State Local

Keep state in the closest common ancestor and avoid lifting it up too early.

```
const [count, setCount] = useState(0);
// Keep it local to the component
// that needs it
```

2 Use State for UI State

Use state for data that affects rendering. Avoid storing derivable values.

```
const [name, setName] = useState('');
const fullName = firstName + lastName;
// Derive instead of storing
```

3 Use Functional Updates

When the new state depends on the previous state, use the functional update form.

```
setCount(prev => prev + 1);
// Safe with multiple updates
```

4 Keep State Shape Simple

Prefer flat state structures. Avoid deeply nested state objects.

```
// ✓ Good
const [user, setUser] = useState({
  name: '', email: ''
});
```

5 Avoid Redundant State

Don't store data that can be calculated from props or other state.

```
// ✗ Avoid
const [total, setTotal] = useState(0);
// Derive it instead
const total = items.reduce(...);
```

6 Group Related State

Group related values together using objects or useReducer.

```
const [form, setForm] = useState({
  name: '', email: ''
});
```

7 Use useReducer

For complex state logic or multiple sub-values, prefer useReducer over useState.

```
const [state, dispatch] =
  useReducer(reducer, initialState);
```

8 Avoid Side Effects in State

Don't run side effects while setting state. Use useEffect instead.

```
// ✗ Avoid
setCount(count + 1);
fetchData();
```

9 Memoize Expensive State

Use useMemo or useCallback to avoid re-creating values or functions unnecessarily.

```
const memoizedValue = useMemo(
  () => compute(data), [data]
);
```

10 Normalize Large State

For large collections, normalize data to make updates easier and faster.

```
// Store by ID for easier updates
const [items, setItems] = useState({
  byId: {}, allIds: []
});
```

Key Takeaway

Good state management leads to predictable UIs, better performance, and easier maintenance.

- Keep state minimal and necessary
- Prefer derivation over duplication
- Structure state for clarity
- Avoid side effects in state updates
- Choose the right tool (useState vs useReducer)

TRY IT YOURSELF — EXERCISE 6: LAB 34 READINESS

Reinforces: a pre-lab self-check confirming your state design is ready before coding in lab34-crm — a readiness capstone.

PRE-LAB READINESS CHECK

Step	What To Do
1 — Teach-Back	Define "controlled input" in two plain sentences, out loud or in notes/lab34-readiness.md.
2 — Evidence Preview	Describe the future evidence: typing updates the name field for Amina without a page reload.
3 — Boundary	State clearly: no API integration happens in the Lab 34 pre-lab -- Lab 35 adds the real Spring Boot call.
4 — Pass Mark	Mark yourself Pass only if Exercise 4's useState TODOs are fully filled in.

READINESS CHECKLIST

Controlled-input definition **Boundary stated** **Pass/Fail marked**

KEY TAKEAWAY TRY IT NOW — Complete Exercise 6 before continuing: exercises/exercise-06-lab34-readiness.md (workspace: examples/module-34-exercises). This is your last checkpoint before the Lab 34 Overview.

LAB OVERVIEW -- STATE AND EVENT MANAGEMENT

Lab 34: React State and Event Management -- lifting the CRM dashboard's state into App (lab34-crm). Theory 1 Hour / Lab 2 Hours / Total 3 Hours.

BUSINESS SCENARIO

- Leadership freezes the gate: no CRM page state ships without immutable updates, derived filters, exclusive form modes, and interaction tests covering create/edit/cancel/search.
- You own that contract for Amina (CUS-1001, ACTIVE), Ravi (CUS-1002, PROSPECT), search "amina", and validation failures on a blank name or a bad email.
- This lifts Lab 33's presentation-only components (CustomerList, CustomerForm, CustomerToolbar) into one App with useState as the single source of truth, ahead of Lab 35's live API wiring.

LEARNING OBJECTIVES

- Store customer records with useState as the single source of truth.
- Create controlled search and form inputs.
- Derive filtered results during render -- no duplicate filtered state.
- Lift selection and form mode into the page with a discriminated union.
- Write immutable create and update handlers.
- Synchronize document.title with useEffect, with cleanup.

WHAT YOU BUILD

- Copy lab33-crm to lab34-crm; lift customers/query/mode/draft/errors into App; control search; derive visible; model mode as closed/create/edit; validate before save; append/replace customers immutably; sync document.title with useEffect; write ≥8 RTL flow tests for create/edit/cancel/search.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor every example; correlation id lab-request-001 appears in create/update/cancel logs.

LAB TASKS AND DELIVERABLES

lab34-crm -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Copy lab33-crm to lab34-crm; lift customers and query into App with useState.
2	Control the search input (value + onChange) wired to CustomerToolbar.
3	Derive visible customers by filtering during render -- never a second useState.
4	Model form mode as a discriminated union: closed / create / edit.
5	Update controlled draft fields immutably; clear only the edited field's error.
6	Validate before saving; block blank name / invalid email with field errors.
7	Append and replace customers immutably (spread) -- never push or mutate in place.
8	Sync document.title with useEffect (with cleanup); write ≥8 RTL flow tests.

EXPECTED DELIVERABLES

- lab34-crm/crm-ui with lifted state CRUD + search.
- Discriminated form modes; immutable updates.
- Client validation with accessible errors.
- Title useEffect with cleanup; no derived-state effects.
- ≥8 RTL interaction tests + green build (twice).
- State notes (docs/state-notes.md) + evidence screenshots.

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs 10

KEY TAKEAWAY In-place mutations or filtered-state effects lose core marks; happy-path-only tests lose automated-verification marks -- prove create, edit, cancel, and search all work.

MODULE 34 SUMMARY

In this module, we learned how to manage state and events in React to build dynamic, interactive, and responsive user interfaces.

KEY CONCEPTS COVERED

Local State (useState)

useState adds state to a functional component; state is local, private, and triggers re-renders.

Handle Events

Respond to user interactions with onClick, onChange, and onSubmit event handlers.

Controlled Forms

Bind value to state and update via onChange, using one state object for related fields.

Lift State Up

Share data via props (down) and callback functions (up); keep data flow unidirectional.

Side Effects (useEffect)

Synchronize with the outside world -- fetches, subscriptions, timers -- and always clean up.

Best Practices

Update state immutably, derive instead of duplicating, and keep state minimal and local.

MODULE FLOW RECAP

1. useState
2. Event Handlers
3. Controlled Forms
4. Lifted State
5. useEffect
6. Lab: lab34-crm

KEY TAKEAWAY Effective state and event management is the foundation of every interactive React application -- state drives the UI, events drive state, and immutability keeps everything predictable.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of React state fundamentals, hooks, re-renders, and controlled inputs.

1. What is the primary purpose of state in React?

A. To store data that never changes

B. To manage dynamic data that affects the UI

C. To handle routing between pages

D. To style components dynamically

2. Which Hook is used to add state to a functional component?

A. useEffect

B. useState

C. useReducer

D. useContext

3. What happens when state is updated in a React component?

A. The DOM is updated immediately

B. The component re-renders with the new state

C. The page reloads

D. React ignores the update

4. Which event is typically used to handle changes in a text input?

A. onClick

B. onChange

C. onSubmit

D. onLoad

KEY TAKEAWAY State manages data that changes over time and drives the UI; useState adds it to a component; updating it re-renders; onChange handles text input changes.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on parent-child data flow, callbacks, functional state updates, and preventing unnecessary re-renders.

5. How do you pass data from a parent component to a child component?

A. Using state

B. Using props

C. Using context only

D. Using localStorage

6. How can a child component send data back to its parent?

A. Using props

B. Using a callback function passed via props

C. Using global variables

D. Using local state

7. What is the recommended way to update state based on the previous state?

A. setCount(count + 1)

B. setCount(prev => prev + 1)

C. setCount({ count: count + 1 })

D. setCount(prev.count + 1)

8. Which of the following helps prevent unnecessary re-renders of components?

A. useState

B. React.memo

C. useEffect

D. setState

KEY TAKEAWAY Props pass data down; a callback prop sends data up; prefer functional updates (prev => prev + 1); React.memo helps prevent unnecessary re-renders.

"Capture the event. Update the state. Let React render the rest."

MODULE 34 COMPLETE

State and Event Management in React

You can now manage local and lifted state with useState, handle user events and controlled forms, synchronize side effects with useEffect, and update state immutably.